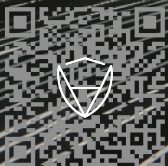




CipheX - Audit

Security Assessment

CertiK Assessed on Dec 11th, 2024





CertiK Assessed on Dec 11th, 2024

CipheX - Audit

The security assessment was prepared by CertiK, the leader in Web3.0 security.

Executive Summary

TYPES

DeFi

ECOSYSTEM

EVM Compatible

METHODS

Manual Review, Static Analysis

LANGUAGE

Solidity

TIMELINE

Delivered on 12/11/2024

KEY COMPONENTS

N/A

CODEBASE

<https://github.com/cytric-io/cyt-25-ciphex->[contracts/tree/4d04720c69dcde1053fe07ef7be4ad2a59ec271c/contract](#)[s](#)

View All in Codebase Page

Highlighted Centralization Risks



Contract upgradeability



Transfers can be paused



Initial owner token share is 100%

Vulnerability Summary



12

Total Findings

8

Resolved

3

Mitigated

0

Partially Resolved

1

Acknowledged

0

Declined



1 Critical

1 Resolved



Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.



3 Major

3 Mitigated



Major risks can include centralization issues and logical errors. Under specific circumstances, these major risks can lead to loss of funds and/or control of the project.



0 Medium

Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.



5 Minor

4 Resolved, 1 Acknowledged



Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.

■ 3 Informational

3 Resolved



Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | CIPHEX - AUDIT

I **Summary**

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

I **Findings**

[CPB-03 : Presale Tokens Are Vulnerable to Zero-ETH Shopping](#)

[CIP-01 : Initial Token Distribution](#)

[CON-01 : Centralized Control of Contract Upgrade](#)

[CON-02 : Centralization Related Risks](#)

[CPB-01 : Deprecated usage of chainlink API `latestAnswer`](#)

[CPB-02 : Arithmetic Precision and Calculation Errors](#)

[CPB-04 : Presale Purchase Restriction Results in Unsold Ciphex Tokens](#)

[CPB-05 : Decimal Mismatch Leading to Incorrect Token Pricing](#)

[CSB-04 : Missing Error Messages to Handle Potential Underflow Cases in `requiredRewards`](#)

[CSB-01 : Early Token Withdrawal Due to Vesting Rate Misconfiguration](#)

[CSB-02 : Inconsistent Vesting Rate](#)

[CSB-03 : Typo](#)

I **Appendix**

I **Disclaimer**

CODEBASE | CIPHEX - AUDIT

Repository

<https://github.com/cytric-io/cyt-25-ciphex-contracts/tree/4d04720c69dcde1053fe07ef7be4ad2a59ec271c/contracts>

AUDIT SCOPE | CIPHEX - AUDIT

3 files audited ● 1 file with Acknowledged findings ● 2 files with Mitigated findings



ID	Repo	File	SHA256 Checksum
● CPB	cytric-io/cyt-25-ciphex-contracts	 CiphexPresale.sol	1f3987b515d592d5ff37476b460e5ece417660aa977d72de6495396c18bebc2d
● CIP	cytric-io/cyt-25-ciphex-contracts	 Ciphex.sol	e2429f2e7763a6f1c3be4e886b13dbd5cd16cb9b0a8f1c20a70b7e7c8bc3384b
● CSB	cytric-io/cyt-25-ciphex-contracts	 CiphexStaking.sol	7178192861abb553c4919ee276fa22443374100858cbc8f1f9e5159a26da455b

APPROACH & METHODS | CIPHEX - AUDIT

This report has been prepared for CipheX to discover issues and vulnerabilities in the source code of the CipheX - Audit project as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Manual Review and Static Analysis techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.

The security assessment resulted in findings that ranged from critical to informational. We recommend addressing these findings to ensure a high level of security standards and industry practices. We suggest recommendations that could better serve the project from the security perspective:

- Testing the smart contracts against both common and uncommon attack vectors;
- Enhance general coding practices for better structures of source codes;
- Add enough unit tests to cover the possible use cases;
- Provide more comments per each function for readability, especially contracts that are verified in public;
- Provide more transparency on privileged activities once the protocol is live.

FINDINGS | CIPHEX - AUDIT



12

Total Findings

1

Critical

3

Major

0

Medium

5

Minor

3

Informational

This report has been prepared to discover issues and vulnerabilities for CipheX - Audit. Through this audit, we have uncovered 12 issues ranging from different severity levels. Utilizing the techniques of Manual Review & Static Analysis to complement rigorous manual code reviews, we discovered the following findings:

ID	Title	Category	Severity	Status
CPB-03	Presale Tokens Are Vulnerable To Zero-ETH Shopping	Logical Issue	Critical	● Resolved
CIP-01	Initial Token Distribution	Centralization	Major	● Mitigated
CON-01	Centralized Control Of Contract Upgrade	Centralization	Major	● Mitigated
CON-02	Centralization Related Risks	Centralization	Major	● Mitigated
CPB-01	Deprecated Usage Of Chainlink API <code>latestAnswer</code>	Volatile Code	Minor	● Resolved
CPB-02	Arithmetic Precision And Calculation Errors	Logical Issue	Minor	● Resolved
CPB-04	Presale Purchase Restriction Results In Unsold Ciphex Tokens	Logical Issue	Minor	● Acknowledged
CPB-05	Decimal Mismatch Leading To Incorrect Token Pricing	Inconsistency	Minor	● Resolved
CSB-04	Missing Error Messages To Handle Potential Underflow Cases In <code>requiredRewards</code>	Logical Issue	Minor	● Resolved
CSB-01	Early Token Withdrawal Due To Vesting Rate Misconfiguration	Design Issue	Informational	● Resolved
CSB-02	Inconsistent Vesting Rate	Inconsistency	Informational	● Resolved

ID	Title	Category	Severity	Status
CSB-03	Typo	Coding Style	Informational	Resolved

CPB-03 | PRESALE TOKENS ARE VULNERABLE TO ZERO-ETH SHOPPING

Category	Severity	Location	Status
Logical Issue	● Critical	CiphexPresale.sol: 181	● Resolved

Description

The `CiphexPresale::buy()` function allows users to pay ETH to purchase Ciphex tokens. When users purchase tokens using ETH (`_token == address(0)`), the contract does not validate that the `_amount` specified by the user is less than or equal to the `msg.value` sent with the transaction. Without this check, attackers can exploit this loophole to acquire presale tokens without making the required payment, draining the entire allocation of presale tokens.

```
1  function buy(  
2      address _token,  
3      uint256 _amount,  
4      address _recipient,  
5      address _referral  
6  ) external payable nonReentrant {  
7      if (_recipient == address(0)) revert ZeroAddress();  
8      if (_amount == 0) revert ZeroValue();  
9      if (_recipient == _referral) revert InvalidReferral();  
10     if (!isPresaleActive()) revert NotActivePresale();  
11     uint256 refundEth;  
12     uint256 tokensAmount;  
13     uint256 _ciphexSupply = ciphexSupply; // gas optimization  
14     if (_token == address(0)) {  
15         if (_amount < msg.value) refundEth = msg.value - _amount;  
16         tokensAmount = _convertEthToTokens(_amount);  
17         if (_ciphexSupply < tokensAmount) {  
18             uint256 delta = tokensAmount - _ciphexSupply;  
19             uint256 ethDelta = convertTokensToEth(delta);  
20             refundEth += ethDelta;  
21             _amount -= ethDelta;  
22             tokensAmount = _ciphexSupply;  
23         }  
24     }  
25 }
```

Proof of Concept

```

it("Audit: buy ciphex token without payment", async () => {
  expect(await presale.isPresaleActive()).to.be.true;
  var balBefore = await ciphex.balanceOf(await staking.getAddress());
  const tx: ContractTransactionResponse = await presale
    .connect(user1)
    .buy(
      ethers.ZeroAddress,
      ethers.parseEther("10"), // <=
      user1.address,
      ethers.ZeroAddress,
      { value: 0 } // <= Pay zero ETH
    );
  expect(tx).to.be.emit(presale, "Bought");
  console.log(await ciphex.balanceOf(await staking.getAddress()) -
    balBefore);
});

```

```

Presale
  buy
    with active presale
      write function buy
261891110000000000000000000000n
  ✓ Audit: buy ciphex token without payment

```

Recommendation

Add a validation check that ensures `msg.value` is equal to or greater than the `_amount` specified by the user.

```
require(msg.value >= _amount, NotEnoughEth());
```

Alleviation

[Ciphex Team, 11/29/2024]: We acknowledge this critical issue and fully agree with CertiK's recommendation to add a validation check ensuring `msg.value >= _amount` for ETH contributions. We have implemented this change in the updated version of our smart contract, ensuring such exploits are impossible. The updated buy function now includes the following check, see link for fix:

<https://github.com/cytric-io/cyt-25-ciphex-contracts/blob/issues/auditReportFixes/contracts/CiphexPresale.sol#L215>

This change effectively mitigates the issue and protects token allocations during the presale.

CIP-01 | INITIAL TOKEN DISTRIBUTION

Category	Severity	Location	Status
Centralization	● Major	Ciphex.sol: 28~29	● Mitigated

Description

All of the `Ciphex` tokens are sent to the `defaultAdmin` addresses. This is a centralization risk because the `defaultAdmin` can distribute tokens without obtaining the consensus of the community. Any compromise to these addresses may allow a hacker to steal and sell tokens on the market, resulting in severe damage to the project.

Recommendation

It is recommended that the team be transparent regarding the initial token distribution process. The token distribution plan should be published in a public location that the community can access. The team should make efforts to restrict access to the private keys of the deployer account or EOAs. A multi-signature ($\frac{2}{3}$, $\frac{3}{5}$) wallet can be used to prevent a single point of failure due to a private key compromise. Additionally, the team can lock up a portion of tokens, release them with a vesting schedule for long-term success, and deanonymize the project team with a third-party KYC provider to create greater accountability.

Alleviation

[Ciphex Team, 11/29/2024]: We acknowledge the risks associated with centralization in token distribution and appreciate CertiK's recommendation. As outlined in Section 3.9 *Creator and Founding Contributor Tokens* and Section 3.2 *Circulating Supply* of the [whitepaper](#), CipheX's initial token distribution plan incorporates stringent safeguards, including multi-sig treasury management protocols:

"CipheX uses multi-sig authentication protocols to manage CPX Tokens or digital currency transfers."

In line with your recommendations:

1. We have minting operations directly associated with the Gnosis Safe multi-signature wallet to prevent single points of failure in the token allocation process.
2. All undistributed tokens are secured within the multi-sig wallet, and detailed token allocation plans have been shared via our Medium article: [Published Token Distribution Plan](#).

[Ciphex Team, 12/10/2024]: The team acknowledged the issue and adopted the multisign solution to ensure the token distribution process at the current stage. The `CPX` contract has transferred all CPX tokens to a Gnosis Safe contract(0xdc75b0c5d94816c27458ed62bec10474a7fc4231) with 3/5 signers in the sensitive function signing process.

- Token transaction hash to the Gnosis Safe Wallet:

<https://etherscan.io/tx/0xcb160fdbf3ec6ef60a6322264fe2329cbede025fa84a7db5d9314dbe5d7d1c42>

- The 5 multisign addresses:

1. eth:0x6d917FD0d699B37E2FfEc9BC6d5c6a163E0B6c2d
2. eth:0xC6bab159Fd9459C12c9C97d6ad57Da872b994893
3. eth:0x52511C1019763864d9e1390cC47b04400C58C795
4. eth:0x22ceb85cdCbab3E4e1CFbE8389F3f7cc7b33FC4C
5. eth:0x2796ddb7A7d2a92D6B199A30A8BD8dFD34d0046F

CON-01 | CENTRALIZED CONTROL OF CONTRACT UPGRADE

Category	Severity	Location	Status
Centralization	● Major	Ciphex.sol: 9; CiphexPresale.sol: 10; CiphexStaking.sol: 8	● Mitigated

Description

The `Ciphex`, `CiphexPresale` and `CiphexStaking` inherit upgradeable contracts, indicating that they are part of an upgradeable system. Upgradeable contracts often pair with a proxy contract that is responsible for managing contract upgrades. The `privileged roles` of the proxy often have the authority to update the implementation contract.

Any compromise of the privileged account could allow a hacker to exploit this authority, potentially altering the implementation contract pointed to by the proxy and thus executing malicious functionality within the implementation contract.

Recommendation

We recommend that the team make efforts to restrict access to the admin of the proxy contract. A strategy of combining a time-lock and a multi-signature (2/3, 3/5) wallet can be used to prevent a single point of failure due to a private key compromise. In addition, the team should be transparent and notify the community in advance whenever they plan to migrate to a new implementation contract.

Here are some feasible short-term and long-term suggestions that would mitigate the potential risk to a different level and suggestions that would permanently fully resolve the risk.

Short Term:

A combination of a time-lock and a multi signature (2/3, 3/5) wallet mitigate the risk by delaying the sensitive operation and avoiding a single point of key management failure.

- A time-lock with reasonable latency, such as 48 hours, for awareness of privileged operations;
AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to a private key compromised;
AND
- A medium/blog link for sharing the time-lock contract and multi-signers addresses information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.

- Provide a link to the **medium/blog** with all of the above information included.

Long Term:

A combination of a time-lock on the contract upgrade operation and a DAO for controlling the upgrade operation mitigate the contract upgrade risk by applying transparency and decentralization.

- A time-lock with reasonable latency, such as 48 hours, for community awareness of privileged operations;
AND
- Introduction of a DAO, governance, or voting module to increase decentralization, transparency, and user involvement;
AND
- A medium/blog link for sharing the time-lock contract, multi-signers addresses, and DAO information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.
- Provide a link to the **medium/blog** with all of the above information included.

Permanent:

Renouncing ownership of the `admin` account or removing the upgrade functionality can *fully* resolve the risk.

- Renounce the ownership and never claim back the privileged role;
OR
- Remove the risky functionality.

Note: we recommend the project team consider the long-term solution or the permanent solution. The project team shall make a decision based on the current state of their project, timeline, and project resources.

I Alleviation

[Ciphex Team, 11/29/2024]: We appreciate CertiK highlighting the centralization risks associated with privileged roles and contract upgrades. As detailed in Section 3.3 Treasury Management Protocols of our [whitepaper](#), Ciphex is committed to mitigating such risks.

To address these concerns, Ciphex has implemented the following measures:

1. Adoption of Multi-Signature Wallets (3/5 model): We have assigned the ownership of critical contracts, including the Ciphex and CiphexStaking contracts, to a Gnosis Safe multi-signature wallet. This significantly reduces the risk of a

single point of failure and ensures all sensitive operations, including pausing transfers, staking rewards, and withdrawing funds, require the consensus of multiple signatories. Multi-signature wallet details have been shared publicly via our Medium article, as per CertiK's recommendations: [Medium Article Here](#)

2. **Timelock Implementation:** A timelock contract has been deployed for critical administrative actions across the Ciphex and CiphexStaking contracts. All privileged operations requiring ownership roles-for example, setting vesting rates or withdrawing staking rewards-are now gated with a 48-hour delay, ensuring enhanced transparency and community awareness. This short-term solution aligns with CertiK's recommendation:
3. [Timelock demonstration](#)
4. Multi-sig Address: 0xDC75B0c5D94816C27458ed62BEc10474A7Fc4231

These changes demonstrate our ongoing commitment to decentralization and the protection of our community's interests. We are confident the combination of timelock mechanisms and multi-signature wallets sufficiently addresses the identified centralization risks. Additionally, we are actively exploring long-term governance solutions like a DAO model for enhanced decentralization.

[Ciphex Team, 12/09/2024]:

The team acknowledged the issue and adopted the multisign solution to ensure the contract upgrade management process at the current stage. The `CiphexPresale` contract has transferred the ownership to a Gnosis Safe contract with 3/5 signers in the sensitive function signing process.

- Grant Role transaction hash for Gnosis Safe:
<https://etherscan.io/tx/0x905901313605ef82df8d1dc0e5b763a31f27db4387781d405d9ff74c06a4232d>
- The 5 multisign addresses:
 1. eth:0x6d917FD0d699B37E2FfEc9BC6d5c6a163E0B6c2d
 2. eth:0xC6bab159Fd9459C12c9C97d6ad57Da872b994893
 3. eth:0x52511C1019763864d9e1390cC47b04400C58C795
 4. eth:0x22ceb85cdCbab3E4e1CFbE8389F3f7cc7b33FC4C
 5. eth:0x2796ddb7A7d2a92D6B199A30A8BD8dFD34d0046F

The `CiphexStaking` contract has transferred the ownership to a 48-hours Timelock contract

- Time-lock contract address: <https://etherscan.io/address/0x567fAb3B2dFa635a4257FF742610D22E92c32400>
- Time lock owner transfer transaction hash:
<https://etherscan.io/tx/0x3820a7a7d0041cba2366bed43d5e21e33e6c6382cc7e020fc764e6eef02f62ac>

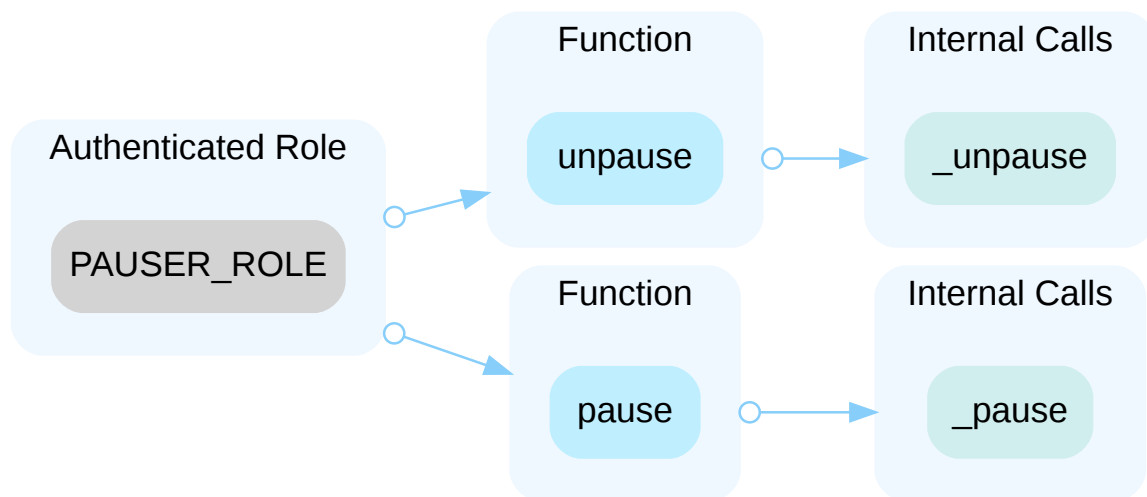
[CertiK, 12/09/2024]: While this strategy has indeed reduced the risk, it's crucial to note that it has not completely eliminated it. CertiK strongly encourages the project team periodically revisit the private key security management of all above-listed addresses.

CON-02 | CENTRALIZATION RELATED RISKS

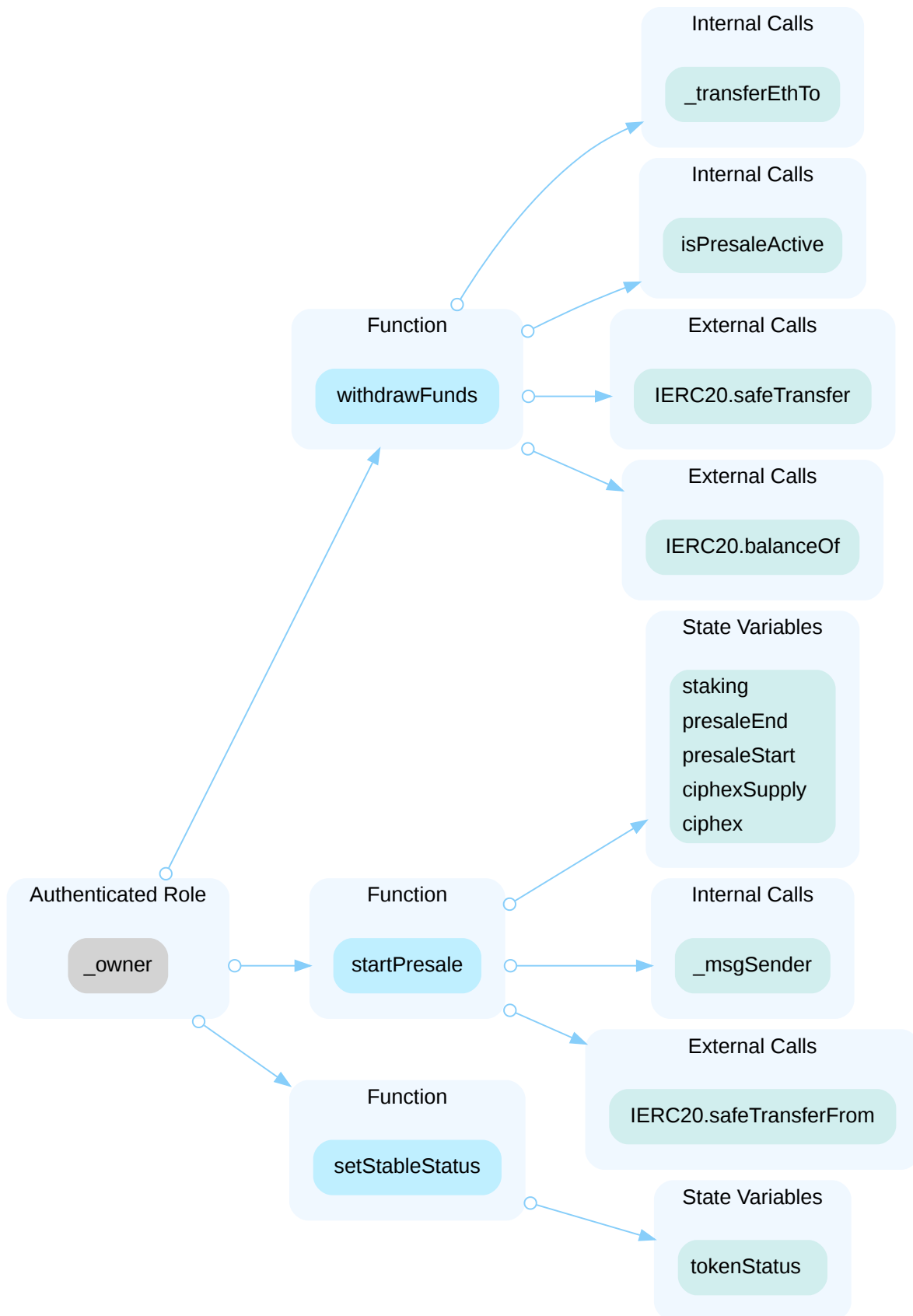
Category	Severity	Location	Status
Centralization	Major	Ciphex.sol: 32, 36; CiphexPresale.sol: 91, 106, 142; CiphexStaking.sol: 88, 104	Mitigated

Description

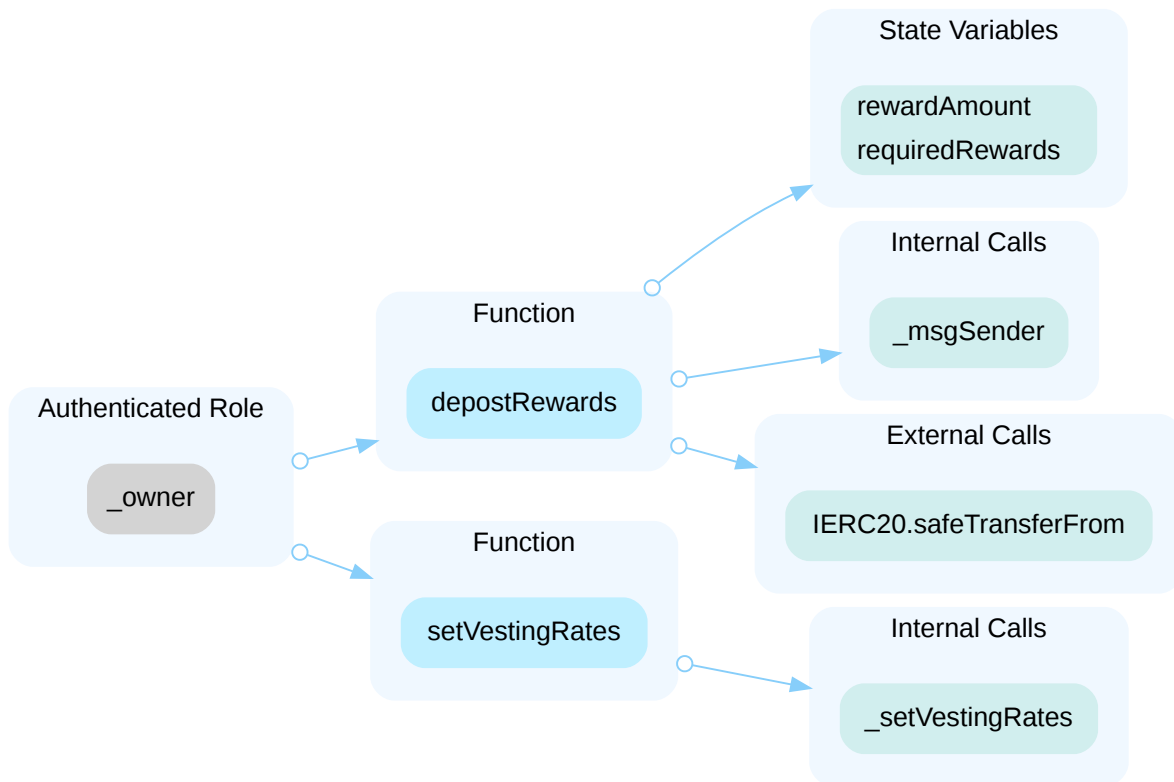
In the contract `Ciphex`, the role `PAUSER_ROLE` has authority over the functions shown in the diagram below. Any compromise to the `PAUSER_ROLE` account may allow the hacker to take advantage of this authority to pause/unpause the token transfer.



In the contract `CiphexPresale`, the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and withdraw specified funds to multiple recipients, start the presale event, and set status for payment tokens.



In the contract `CiphexStaking`, the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and deposit rewards, set vesting rates.



Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (2/3, 3/5) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
OR
- Remove the risky functionality.

I Alleviation

[Ciphex Team, 11/29/2024]: We appreciate CertiK highlighting the centralization risks associated with privileged roles and contract upgrades. As detailed in Section 3.3 Treasury Management Protocols of our [whitepaper](#), Ciphex is committed to mitigating such risks.

To address these concerns, Ciphex has implemented the following measures:

1. Adoption of Multi-Signature Wallets (3/5 model): We have assigned the ownership of critical contracts, including the Ciphex and CiphexStaking contracts, to a Gnosis Safe multi-signature wallet. This significantly reduces the risk of a single point of failure and ensures all sensitive operations, including pausing transfers, staking rewards, and withdrawing funds, require the consensus of multiple signatories. Multi-signature wallet details have been shared publicly via our Medium article, as per CertiK's recommendations: [Medium Article Here](#)
2. Timelock Implementation: A timelock contract has been deployed for critical administrative actions across the Ciphex and CiphexStaking contracts. All privileged operations requiring ownership roles-for example, setting vesting rates or withdrawing staking rewards-are now gated with a 48-hour delay, ensuring enhanced transparency and community awareness. This short-term solution aligns with CertiK's recommendation:
3. [Timelock demonstration](#)
4. Multi-sig Address: 0xDC75B0c5D94816C27458ed62BEc10474A7Fc4231

These changes demonstrate our ongoing commitment to decentralization and the protection of our community's interests. We are confident the combination of timelock mechanisms and multi-signature wallets sufficiently addresses the identified centralization risks. Additionally, we are actively exploring long-term governance solutions like a DAO model for enhanced decentralization.

[Ciphex Team, 12/09/2024]:

The team acknowledged the issue and adopted the multisign solution to ensure the contract privileged roles management at the current stage. The `CiphexPresale` contract has transferred the ownership to a Gnosis Safe contract with 3/5 signers in the sensitive function signing process.

- Grant Role transaction hash for Gnosis Safe:
<https://etherscan.io/tx/0x905901313605ef82df8d1dc0e5b763a31f27db4387781d405d9ff74c06a4232d>
- The 5 multisign addresses:
 1. eth:0x6d917FD0d699B37E2FEc9BC6d5c6a163E0B6c2d
 2. eth:0xC6bab159Fd9459C12c9C97d6ad57Da872b994893
 3. eth:0x52511C1019763864d9e1390cC47b04400C58C795
 4. eth:0x22ceb85cdCbab3E4e1CFbE8389F3f7cc7b33FC4C
 5. eth:0x2796ddb7A7d2a92D6B199A30A8BD8dFD34d0046F

The `CiphexStaking` contract has transferred the ownership to a 48-hours Timelock contract

- Time-lock contract address: <https://etherscan.io/address/0x567fAb3B2dFa635a4257FF742610D22E92c32400>
- Time lock owner transfer transaction hash:
<https://etherscan.io/tx/0x3820a7a7d0041cba2366bed43d5e21e33e6c6382cc7e020fc764e6eef02f62ac>

[CertiK, 12/09/2024]: While this strategy has indeed reduced the risk, it's crucial to note that it has not completely eliminated it. CertiK strongly encourages the project team periodically revisit the private key security management of all above-listed addresses.

CPB-01 | DEPRECATED USAGE OF CHAINLINK API `latestAnswer`

Category	Severity	Location	Status
Volatile Code	● Minor	CiphexPresale.sol: 342~344	● Resolved

Description

The Chainlink API (`latestAnswer`) used in the Presale contract is deprecated:

This API is deprecated. Please see API Reference for the latest Price Feed API.

Reference: <https://web.archive.org/web/20210304160150/https://docs.chain.link/docs/deprecated-aggregatorinterface-api-reference>.

This function does not error if no answer has been reached but returns 0. Besides, the `latestAnswer` is reported with 18 decimals for crypto quotes but 8 decimals for FX quotes (See Chainlink FAQ for more details). A best practice is to get the decimals from the oracles instead of hard-coding them in the contract.

Recommendation

Use the `latestRoundData` function to get the price instead. Add checks on the return data with proper revert messages if the price is stale or the round is incomplete.

Alleviation

[Ciphex Team, 12/03/2024]: The team heeded the advice and resolved the issue in this [commit](#).

CPB-02 | ARITHMETIC PRECISION AND CALCULATION ERRORS

Category	Severity	Location	Status
Logical Issue	● Minor	CiphexPresale.sol: 258~263, 273~276	● Resolved

Description

The linked function performs complex arithmetic operations involving multiple constants and variables with different decimal places. There is inconsistent use of PRECISION, which can lead to incorrect token pricing and conversion calculations.

```
252     function getTokenPrice() public view returns (uint256) {
253         if (!isPresaleActive()) return 0;
254         uint256 deltaTime = block.timestamp - presaleStart;
255         uint256 periods = deltaTime / MULTIPLIER_ITERATION;
256         if (periods == 0) {
257             return START_PRICE;
258         } else {
259             uint256 price = START_PRICE +
260                 (((START_PRICE * PRICE_MULTIPLIER * PRECISION) / PRECISION) /
261                 PRECISION) * periods);
262             if (price > MAX_PRICE) return MAX_PRICE;
263             return price;
264         }
265     }
```

The calculation can be simplified and corrected to:

```
uint256 price = START_PRICE + ((START_PRICE * PRICE_MULTIPLIER * periods) /
PRECISION);
```

The unnecessary multiplication and division by PRECISION can cause rounding issue and affect calculation precision.

Users might be able to exploit these inaccuracies to purchase tokens at unintended prices, resulting in financial loss to the contract or other participants.

Recommendation

It's recommended the team simplify calculations by canceling out redundant multiplications and divisions.

Alleviation

[Ciphex Team, 12/03/2024]: The team heeded the advice and resolved the issue in this [commit](#).

CPB-04 | PRESALE PURCHASE RESTRICTION RESULTS IN UNSOLD CIPHEX TOKENS

Category	Severity	Location	Status
Logical Issue	Minor	CiphexPresale.sol: 222	Acknowledged

Description

In the `buy()` function, there's a logical flaw that prevents completing a pre-sale purchase when the remaining Ciphex token amount is neither zero nor smaller than `MIN_TOKENS_TO_BUY`:

```
219     uint256 supplyAfterBuy = _ciphexSupply - tokensAmount;
220     if (
221         tokensAmount < MIN_TOKENS_TO_BUY ||
222         (supplyAfterBuy < MIN_TOKENS_TO_BUY && supplyAfterBuy != 0) // <=
223     ) revert NotEnoughCiphex();
```

This logic check causes issues when nearing the end of the pre-sale. For instance:

1. If there are `MIN_TOKENS_TO_BUY * 1.5` tokens left in the presale contract.
2. A buyer attempts to purchase `MIN_TOKENS_TO_BUY` tokens.
3. As `supplyAfterBuy = MIN_TOKENS_TO_BUY * 0.5`, the transaction reverts due to the check, even though there are sufficient tokens available.

The contract should allow the buyer to purchase any amount of tokens between `MIN_TOKENS_TO_BUY` and `MIN_TOKENS_TO_BUY * 1.5`.

Recommendation

It's recommended that the team remove the restrictive check for `supplyAfterBuy` in the `buy()` function.

Alleviation

[Ciphex Team, 11/29/2024]: The team acknowledged the finding and decided not to change the current codebase.

CPB-05 | DECIMAL MISMATCH LEADING TO INCORRECT TOKEN PRICING

Category	Severity	Location	Status
Inconsistency	Minor	CiphexPresale.sol: 206~217, 335~336	Resolved

Description

The user-provided `_amount` is assumed to be in USD and converted to Ciphex tokens through the `_convertUsdToTokens()` function. The contract assumes a decimal standard of 6 for payments.

```
206     tokensAmount = _convertUsdToTokens(_amount); // <= _amount of USD
207     if (_ciphexSupply < tokensAmount) {
208         uint256 delta = tokensAmount - _ciphexSupply;
209         uint256 usdDelta = convertTokensToUsd(delta);
210         _amount -= usdDelta;
211         tokensAmount = _ciphexSupply;
212     }
213     IERC20(_token).safeTransferFrom(
214         _msgSender(),
215         address(this),
216         _amount // <= _amount of USDT or USDC
217     );
```

The decimals of token price (USD) is 6, which can be derived from:

```
29     uint256 public constant USD = 1e6;
```

While the contract's fixed decimal handling works correctly for USDT/USDC with 6 decimals, it can lead to significant conversion errors with other implementations. For example, certain bridged token versions like the USDT on Binance Smart Chain (BSC-USDT) have 18 decimals. This discrepancy can result in incorrect conversion rates and therefore incorrect pricing of Ciphex tokens.

Recommendation

Ensure that payment tokens with decimals other than 6 are disallowed.

Alleviation

[Ciphex Team, 11/29/2024]: The team heeded the advice and resolved the issue in commit: <https://github.com/cytrix-io/cyt-25-ciphex-contracts/commit/79ece0affc9e0ece6fba3780cc685866646220ff>. To prevent errors in token conversion rates, we

have updated our implementation to enforce a strict decimal check for payment tokens. Only tokens with a standard of 6 decimals (such as USDC and USDT) are now allowed for participation in the CiphexPresale contract.

CSB-04 | MISSING ERROR MESSAGES TO HANDLE POTENTIAL UNDERFLOW CASES IN `requiredRewards`

Category	Severity	Location	Status
Logical Issue	Minor	CiphexStaking.sol: 91	Resolved

Description

The `depostRewards` function allows the owner to deposit rewards into the contract and updates the `requiredRewards` variable. If the owner attempts to deposit more rewards than the current `requiredRewards`, the subtraction `requiredRewards -= _amount` can cause an underflow, reverting the transaction. Besides, there is no explicit check or error message to inform the owner that the deposit amount exceeds the `requiredRewards`, making it unclear why the transaction failed.

Recommendation

It's recommend the team implement proper error handling logic to deal with the aforementioned transaction revert case.

Alleviation

[Ciphex Team, 12/09/2024]: The team heeded the advice and resolved the issue in this [commit](#).

CSB-01 | EARLY TOKEN WITHDRAWAL DUE TO VESTING RATE MISCONFIGURATION

Category	Severity	Location	Status
Design Issue	● Informational	CiphexStaking.sol: 213~214, 252~254	● Resolved

Description

In the `_calculateWithdrawableAmount()` function, the amount of tokens withdrawable by a user is calculated using the formula: `remainder * vestingRates[vestingMonth - 1] / MULTIPLIER`.

```
252     return
253         ((remainder * vestingRates[vestingMonth - 1] * MULTIPLIER) /
254          MULTIPLIER) / MULTIPLIER;
```

The `_setVestingRates()` anticipates that the last vesting rate (`_vestingRates[_vestingRates.length - 1]`) is set to exactly `MULTIPLIER`:

```
211     function _setVestingRates(uint256[] calldata _vestingRates) internal {
212         if (_vestingRates.length != 12) revert InvalidVestingRatesLengh();
213         if (_vestingRates[_vestingRates.length - 1] != MULTIPLIER) // <=
214             revert InvalidVestingRate();
```

This configuration results in an issue: during the last vesting month, the withdrawable token amount simplifies to `remainder`, effectively allowing users to withdraw all their tokens one month before the completion of the full `VESTING_PERIOD`.

Recommendation

It's recommended that the team consider removing the aforementioned check.

Alleviation

[Ciphex Team, 11/29/2024]: The described functionality is deliberate and adheres to our design intent, as stated in the project's requirements and Section 3.11 *Vesting Schedules* of the [whitepaper](#). Specifically:

"Vesting periods are structured dynamically, with each vesting rate proportional to the remaining token balance."

The functionality ensures that users are able to withdraw their full balance at the conclusion of their vesting period, and this final month of behavior matches our objective to simplify the user experience as vesting concludes. Requiring additional manual configurations for the last vesting month would introduce unnecessary complexities and increase computational overhead without additional security benefits.

Accordingly, we believe this issue should be reclassified as design-intended and marked resolved.

CSB-02 | INCONSISTENT VESTING RATE

Category	Severity	Location	Status
Inconsistency	● Informational	CiphexStaking.sol: 253~254	● Resolved

Description

The formula to calculate withdrawable token amount is `remainder * vestingRates[vestingMonth - 1] / MULTIPLIER`. This formula results in different vesting outcomes depending on previous withdrawals.

For example: A user initially stakes `1000e18` tokens. At the midpoint of the vesting period, the intended vesting rate is 50%.

- Conversely, if the user has not made any prior withdrawals, the formula calculates `1000e18 * 50% = 500e18`.
- If the user has previously withdrawn `300e18` vested tokens, the formula calculates the withdrawable amount as `(1000e18 - 300e18) * 50% = 350e18`. Adding the previously withdrawn tokens, the total vested tokens become `650e18`, which effectively accelerates the vesting rate.

As a result, the intended vesting schedule is not adhered to, leading to a different vesting experience.

Recommendation

To ensure a consistent vesting rate irrespective of the user's withdrawal history, the contract should be modified to apply the vesting rate to the total initially staked tokens rather than the remainder.

Alleviation

[Ciphex Team, 11/29/2024]: Thank you for noting this observation; however, we would like to clarify that the behavior described is by design and aligns with our vesting logic outlined in Section 3.11 *Vesting Schedules* of the [whitepaper](#). The Ciphex vesting protocol calculates withdrawable amounts dynamically based on the remaining staked tokens. This ensures that users who withdraw early have a predictable experience, and any withdrawals made before full vesting are aligned with their remaining balance, as described in the whitepaper:

"CPX Tokens will automatically enter a one-year vesting schedule once their six-month lockup period ends. In the first month, 3% of CPX Tokens will vest and become unrestricted, followed by increasing release rates until all CPX Tokens are fully vested within 12 months."

As such, the total vested amounts are contingent on individual withdrawal patterns, ensuring fairness and consistency in the vesting experience. This design enables flexibility while protecting the integrity of the vesting pool and its proportional token distribution.

CSB-03 | TYPO

Category	Severity	Location	Status
Coding Style	● Informational	CiphexStaking.sol: 88	● Resolved

Description

The `depostRewards` should be `depositRewards`.

Recommendation

It's recommended the team correct the typo.

Alleviation

[Ciphex Team, 11/29/2024]: The team heeded the advice and resolved the issue in commit: <https://github.com/cytric-io/cyt-25-ciphex-contracts/commit/79ece0affc9e0ece6fba3780cc685866646220ff>.

APPENDIX | CIPHEX - AUDIT

Finding Categories

Categories	Description
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.

Checksum Calculation Method

The "Checksum" field in the "Audit Scope" section is calculated as the SHA-256 (Secure Hash Algorithm 2 with digest size of 256 bits) digest of the content of each file hosted in the listed source repository under the specified commit.

The result is hexadecimal encoded and is the same as the output of the Linux "sha256sum" command against the target file.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevating Your Entire **Web3** Journey

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is a leading blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.

